

spaces in the `unvisitedNeighbors` variables. Four `if` statements check that the current `x`, `y` position is not on the border of the maze (so that we still have a neighboring space to check) and whether the neighboring space's `x`, `y` coordinates don't appear in the `hasVisited` list or array already.

If all the neighbors have been visited, the function returns so that it can backtrack to an earlier space. The Python code continues on to check for the base case:

Python

```
if len(unvisitedNeighbors) == 0:
    # BASE CASE
    # All neighboring spaces have been visited, so this is a
    # dead end. Backtrack to an earlier space:
    return
```

The JavaScript code does so as follows:

JavaScript

```
if (unvisitedNeighbors.length === 0) {
    // BASE CASE
    // All neighboring spaces have been visited, so this is a
    // dead end. Backtrack to an earlier space:
    return;
```

The base case for the recursive backtracking algorithm occurs when no unvisited neighbors remain to visit next. In this case, the function simply returns. The `visit()` function itself has no return value. Rather, the recursive function calls `visit()` to modify the maze data structure in the global `maze` variable as a side effect. When the original function call to `maze()` returns, the `maze` global variable contains the completely generated maze.

The Python code continues on to the recursive case like this:

Python

```
else:
    # RECURSIVE CASE
    # Randomly pick an unvisited neighbor to visit:
    nextIntersection = random.choice(unvisitedNeighbors)
```
